[디버깅] 메모리 누수를 점검하는 방법 - CRTDBG (예제 포함)

2020.04.11 23:12 Programming/Debug…

메모리 누수와 관련된 포스팅은 아래 링크를 참고해주세요.

[디버깅] 메모리 누수와 점검하는 방법 소개

[디버깅] 메모리 …

소개 Unmanaged Code인
C++은 메모리 할당과 해제…

luckygg.tistory.com

MFC Dialog 예시

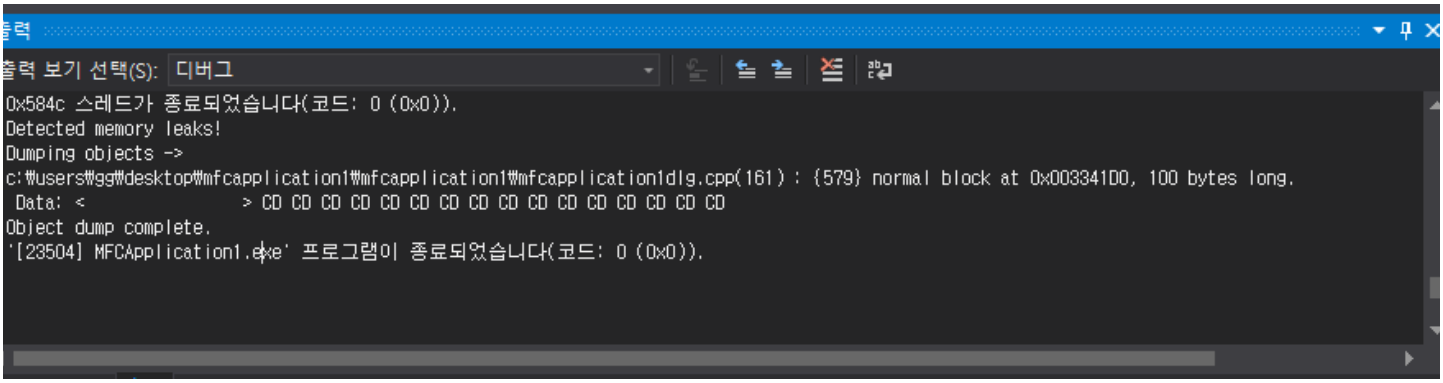
MFC Dialog 프로젝트 예제를 통해 고의로 메모리 누수를 발생시켜 보겠습니다. Main Dialog에 다음과 같이 멤버 변수로 `byte`형 포인터를 선언합니다.

```
byte* m_pBuffer;
```

그리고 버튼을 추가하고, 버튼 클릭 시 다음과 같이 100byte를 동적 할당 하도록 코드를 수정합니다.

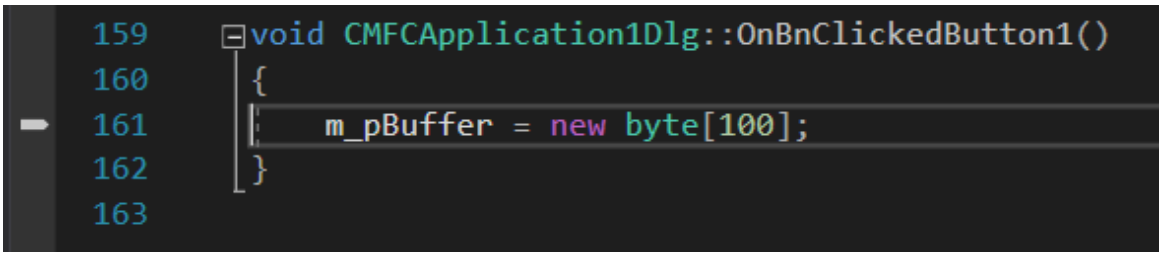
```
void CMFCApplication1Dlg::OnBnClickedButton1()
{
m_pBuffer = new byte[100];
}
```

F5로 디버그 모드로 실행 후 버튼을 클릭하고 프로그램을 종료하면, 출력 창에 다음과 같이 Memory Leak 메시지가 표시됩니다.



Memory Leak이 감지된 메시지 창

이 출력 창에서 `F4` 또는 메시지를 `더블 클릭`하면, 소스 코드의 어느 부분에서 메모리 누수가 발생했는지 표시해줍니다.



메모리 누수가 감지된 부분이 표시된 모습

그럼, 출력 창에 표시된 메모리 누수 메시지를 분석해 보겠습니다. `Detected memory leaks!` 메시지 이후에 `xxx.cpp(161)`은 해당 경로의 소스 파일의 **161 라인에서 누수가 발생**했다는 의미입니다. 그리고 `{579}`는 메모리 할당 순서를 의미하며, **579번째 할당된 부분에 누수가 발견**됐다는 뜻입니다. `0x01602760`는 메모리 주소이고, `100 bytes long`은 누수된 크기입니다.

이렇게 MFC 프로젝트에서는 친절하게 소스 코드 파일과 라인까지 출력되어 편하게 디버깅이 가능합니다. 그 이유는 `xxxDlg.cpp`의 상단에 아래처럼 정의가 되어있기 때문입니다.

```
#ifdef _DEBUG
#define new DEBUG_NEW
#endif
```

위 코드의 의미는, Debug 모드라면 `new`를 `DEBUG_NEW`로 사용하겠다는 의미입니다. `DEBUG_NEW`는 `afx.h`에 아래처럼 정의되어 있습니다.

```
#define THIS_FILE __FILE__
```

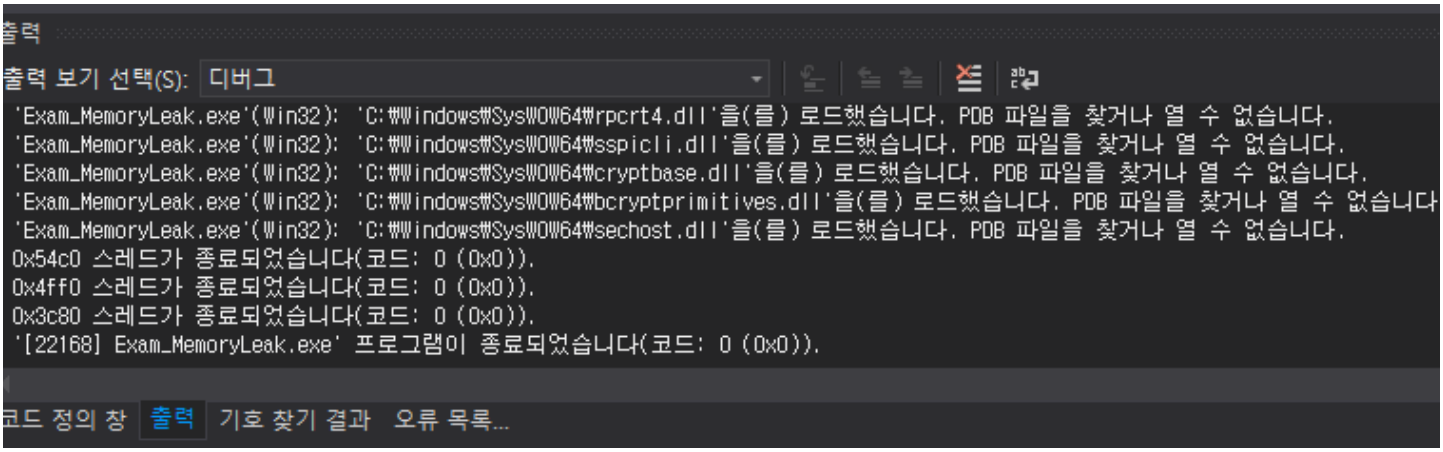
```
//...
#define DEBUG_NEW new(THIS_FILE, __LINE__)
```

C++ Console 예시

이번에는 **C++ Console 프로젝트**에서 동일한 방법으로 테스트를 해보겠습니다. `main()`에 다음처럼 100byte를 동적 할당합니다.

```
int main()
{
byte* buff = new byte[100];
return 0;
}
```

그리고 실행 후 종료하면 출력 창에 별다른 메시지가 없습니다. 메모리 누수가 없는것 같죠?

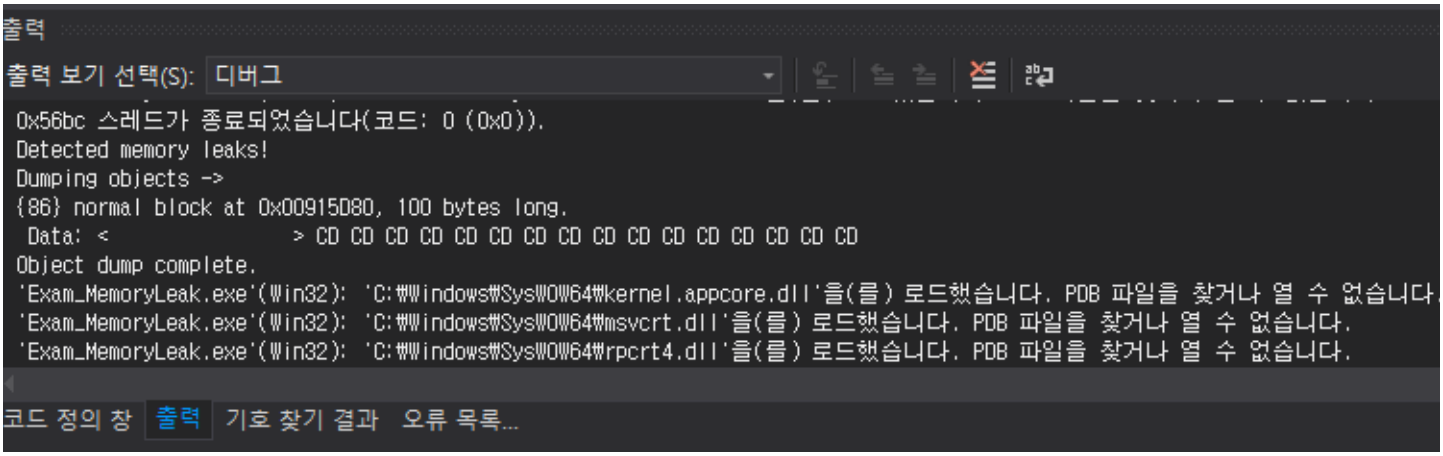


디버그 실행 결과 화면

메모리 누수가 없는지 점검하기 위해 아래처럼 코드를 수정합니다.

```
#include "pch.h"
#include <iostream>
#include <Windows.h>
#define _CRTDBG_MAP_ALLOC
#include <cstdlib>
#include <crtdbg.h>
int main()
{
byte* buff = new byte[100];
_CrtDumpMemoryLeaks();
return 0;
}
```

그리고 실행 후 종료하면 메시지 창에 `Detected memory leaks!` 메시지가 출력됩니다.

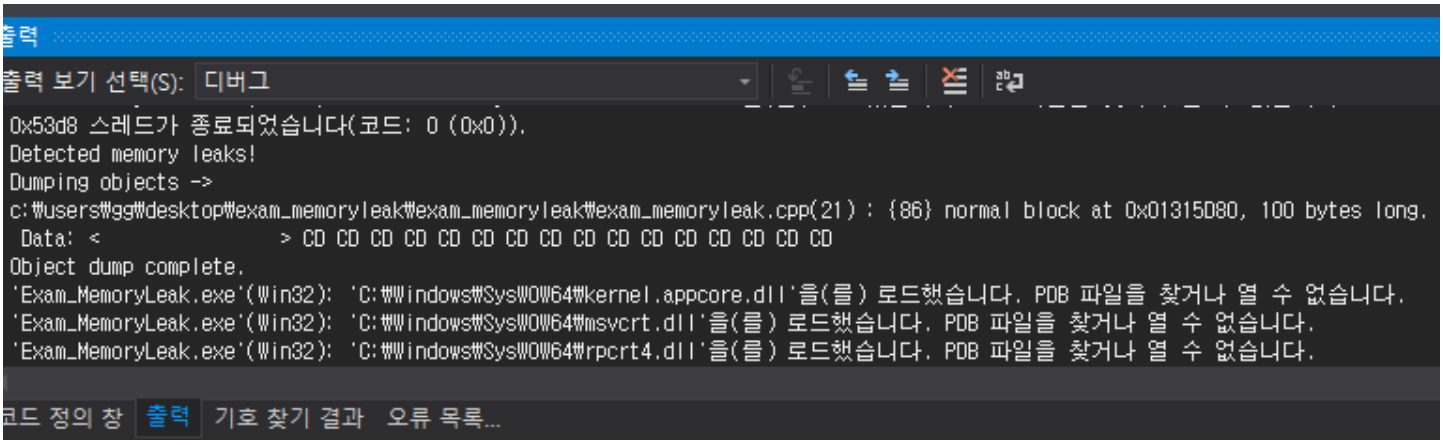


Memory Leak이 감지된 메시지 창

MFC와는 다르게 **소스 파일**과 **라인 수**가 표시되지 않는 차이점이 있습니다. 이번에는 아래처럼 코드를 변경하여 테스트해 보겠습니다.

```
#include "pch.h"
#include <iostream>
#include <Windows.h>
#define _CRTDBG_MAP_ALLOC
#include <cstdlib>
#include <crtdbg.h>
#ifdef _DEBUG
#define new new ( _NORMAL_BLOCK , __FILE__ , __LINE__ )
#endif
int main()
{
byte* buff = new byte[100];
_CrtDumpMemoryLeaks();
return 0;
}
```

변경된 소스 코드 실행 후 결과는 다음과 같습니다.

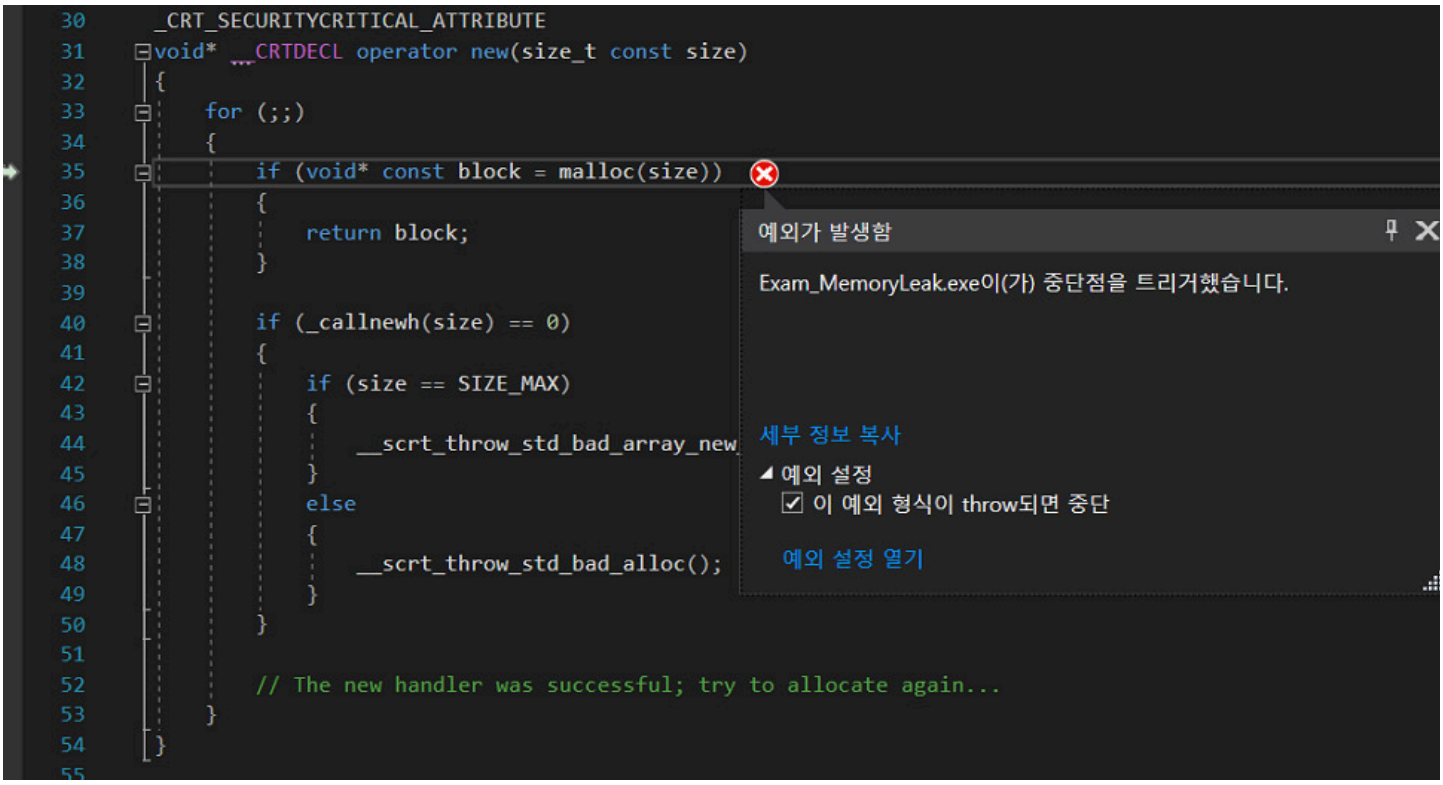


소스 파일과 라인이 표시된 메모리 누수 메시지

이제 소스 파일과 라인 수가 표시되고, `F4` 또는 `더블 클릭`으로 메모리 누수 지점 확인이 가능해집니다. MFC와 마찬가지로, `new`를 `new ( _NORMAL_BLOCK , __FILE__ , __LINE__ )`로 치환하여 세부 내용 출력이 가능한 것입니다.

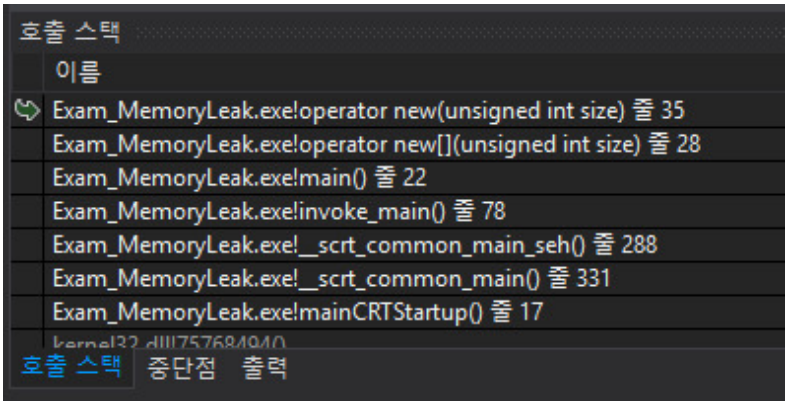
여기서 한 가지 더 알려드릴 내용이 있습니다. 위 메모리 누수 메시지에 `{86}`이 있는데요. 이는 86번째 메모리 할당에서 문제가 있다는 의미입니다. `main()` 첫 부분에 `_CrtSetBreakAlloc(86)`을 추가하면, 86번째 메모리 할당 지점에서 실행이 중지됩니다.

```
#include "pch.h"
#include <iostream>
#include <Windows.h>
#define _CRTDBG_MAP_ALLOC
#include <cstdlib>
#include <crtdbg.h>
#ifdef _DEBUG
#define new new ( _NORMAL_BLOCK , __FILE__ , __LINE__ )
#endif
int main()
{
_CrtSetBreakAlloc(86);
byte* buff = new byte[100];
_CrtDumpMemoryLeaks();
return 0;
}
```



예외가 발생한 화면

호출 스택(Call Stack)을 사용하여 호출 순서를 살펴보겠습니다.



호출 스택 화면

세 번째에 Exam_MemoryLeak.exe!main() 줄 22가 보입니다. 더블 클릭하면 아래의 화면이 표시됩니다.

Break가 걸린 지점

하지만 이 방법은 조금 애매합니다. MFC에서 동일하게 테스트한 결과 메모리 할당 순서가 실행 때마다 바뀌어 적용이 안 됩니다. 하지만 C++ Console은 항상 일정하더군요. 이 부분은 아직 의문이지만, 굳이 필요하진 않을 것 같네요.

마지막으로, 프로그램 종료 부분에 _CrtDumpMemoryLeaks()를 넣는 것 대신, 첫 부분에 _CrtSetDbgFlag(_CRTDBG_ALLOC_MEM_DF | _CRTDBG_LEAK_CHECK_DF)를 추가하는 방법이 있습니다.

```
#include "pch.h"
#include <iostream>
#include <Windows.h>
#define _CRTDBG_MAP_ALLOC
#include <cstdlib>
#include <crtdbg.h>
#ifdef _DEBUG
#define new new ( _NORMAL_BLOCK , __FILE__ , __LINE__ )
#endif
int main()
```

```
{
_CrtSetDbgFlag(_CRTDBG_ALLOC_MEM_DF | _CRTDBG_LEAK_CHECK_DF);
byte* buff = new byte[100];
//_CrtDumpMemoryLeaks();
return 0;
}
```

마찬가지로, 메시지 출력 창에 소스 파일과 라인 수가 표시되어 메모리 누수 지점을 찾을 수 있습니다. 즉, 지금까지 설명한 것을 토대로, 최종적으로 위 예제 코드를 적용하면 되겠습니다.